



Alumnos:

Jonathan Roberto Acosta López SMIS099321

Rene Alexander Araujo Soto SMIS060720

Mario Alexander Hernández Quevedo SMIS075121

Carrera:

Ingeniería en sistemas y redes informáticas

Docente:

Ing. Marco Antonio Arévalo Zambrano

Asignatura:

Desarrollo de aplicaciones con inteligencia artificial

Modulo: 04-2026

Descripción del servicio y endpoint crítico

Servicio: API RESTful FastAPI que expone capacidades de IA (evaluación de código, generación de cursos, chat educativo).

Endpoint crítico seleccionado: POST /api/evaluate

Razón: Es el endpoint de mayor uso, orquesta el pipeline completo de 5 fases con Gemini y análisis estático.

Stack: Python 3.11 · FastAPI · SQLite · Google Gemini · Uvicorn

Preguntas de observabilidad y campos registrados

Preguntas que responden los logs:

- 1. ¿Qué request llegó y cuándo?
- 2. ¿Qué endpoint ejecutó y con qué parámetros?
- 3. ¿Cuánto tardó en responder?
- 4. ¿Qué modelo de IA procesó la solicitud?
- 5. ¿Qué tipo de error ocurrió si falló?

Campo	Descripción	Ejemplo
request_id	UUID corto (8 chars) — correlaciona todos los logs de una solicitud	a3f2b1c9
method	Método HTTP	POST
path	Ruta ejecutada	/api/evaluate
status	Código HTTP de respuesta	200
duration_ms	Duración total en milisegundos	3.42
language	Lenguaje evaluado (solo /api/evaluate)	python
len	Longitud del código recibido (chars)	42
timestamp	Fecha y hora UTC ISO-8601	2026-08-08T14:32:01

Datos excluidos por privacidad/seguridad: API keys de Gemini, tokens de sesión, contraseñas, código fuente completo del usuario en producción.

Código de instrumentación

```
class RequestIDMiddleware(BaseHTTPMiddleware):  
    async def dispatch(self, request: Request, call_next):  
        request_id = str(uuid.uuid4())[:8]  
        request.state.request_id = request_id  
        start = time.time()  
        response = await call_next(request)  
        duration_ms = round((time.time() - start) * 1000, 2)  
        response.headers["X-Request-ID"] = request_id  
        response.headers["X-Response-Time-Ms"] = str(duration_ms)  
        logger.info(  
            "request_id=%s method=%s path=%s status=%d duration_ms=%s",  
            request_id, request.method, request.url.path,  
            response.status_code, duration_ms,  
        )  
        return response
```

Y el log de un endpoint:

```
logger.info(  
    "evaluate_code — request_id=%s language=%s len=%d",  
    request_id, payload.language, len(payload.code),  
)
```

```
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000

2026-08-13 21:14:28,616 [INFO] eduia.api: request_id=129dcfc3 method=GET path=/health status=200 duration_ms=8.69
INFO: 127.0.0.1:53550 - "GET /health HTTP/1.1" 200 OK

2026-08-13 21:14:28,919 [INFO] eduia.api: evaluate_code - request_id=31fe06ea language=python len=29
2026-08-13 21:14:28,920 [INFO] eduia.api: request_id=31fe06ea method=POST path=/api/evaluate status=503 duration_ms=2.2
INFO: 127.0.0.1:53551 - "POST /api/evaluate HTTP/1.1" 503 Service Unavailable

2026-08-13 21:14:29,223 [INFO] eduia.api: request_id=9f650024 method=POST path=/api/evaluate status=422 duration_ms=0.71
INFO: 127.0.0.1:53552 - "POST /api/evaluate HTTP/1.1" 422 Unprocessable Content

2026-08-13 21:14:29,527 [INFO] eduia.api: request_id=38b0af42 method=GET path=/metadata status=200 duration_ms=0.69
INFO: 127.0.0.1:53553 - "GET /metadata HTTP/1.1" 200 OK

2026-08-13 21:14:29,830 [INFO] eduia.api: request_id=0129ad52 method=GET path=/api/stats status=200 duration_ms=0.64
INFO: 127.0.0.1:53554 - "GET /api/stats HTTP/1.1" 200 OK
```

Evidencia solicitud exitosa

POST

/api/courses/generate

Genera la estructura de un curso personalizado con IA

Generador de cursos personalizados — usa `AIManager.generate_course_topics_structure` de `utils_ai.py`.

Genera una estructura de N temas adaptada al lenguaje y nivel del estudiante. Si Gemini está disponible, enriquece descripciones y objetivos con IA; si no, retorna la estructura predefinida de calidad garantizada.

Validaciones de entrada

Campo	Regla	Error
language	Valor en lista soportada	422
level	principiante / intermedio / avanzado	422
sections_count	Entre 1 y 10	422

Parameters

No parameters

Cancel

Request body required

application/json

Edit Value | Schema

```
{
  "language": "python",
  "level": "principiante",
  "sections_count": 5
}
```

Execute

Clear

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/courses/generate' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "language": "python",
    "level": "principiante",
    "sections_count": 5
  }'
```

Request URL

http://127.0.0.1:8000/api/courses/generate

Server response

CodeDetails

200

Response body

```
{
  "language": "python",
  "level": "principiante",
  "sections_count": 5,
  "topics": [
    {
      "topic_number": 1,
      "title": "Instalaci3n, sintaxis b3sica, variables y tipos",
      "description": "Aprende instalaci3n, sintaxis b3sica, variables y tipos en Python",
      "objectives": "Dominar instalaci3n, sintaxis b3sica, variables y tipos aplicado a Python",
      "estimated_hours": 1,
      "order_index": 0,
      "order": 1
    },
    {
      "topic_number": 2,
      "title": "Operadores, entrada/salida, primeros programas",
      "description": "Aprende operadores, entrada/salida, primeros programas en Python",
      "objectives": "Dominar operadores, entrada/salida, primeros programas aplicado a Python",
      "estimated_hours": 1,
      "order_index": 1,
      "order": 2
    },
    {
      "topic_number": 3,
      "title": "Condicionales y bucles b3sicos",
      "description": "Aprende condicionales y bucles b3sicos en Python",
      "objectives": "Dominar condicionales y bucles b3sicos aplicado a Python",
      "estimated_hours": 1,
      "order_index": 2,
      "order": 3
    }
  ]
}
```

Response headers

```
content-length: 1489
content-type: application/json
date: Fri, 14 Aug 2026 03:23:48 GMT
server: uvicorn
x-request-id: 83c7ac2d
x-response-time-ms: 1.37
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header

Example Value | Schema

```
{
  "language": "string",
  "level": "string",
  "sections_count": 0,
  "topics": [
    {
      "topic_number": 1,
      "title": "Instalaci3n, sintaxis b3sica, variables y tipos",
      "description": "Aprende instalaci3n, sintaxis b3sica, variables y tipos en Python",
      "objectives": "Dominar instalaci3n, sintaxis b3sica, variables y tipos aplicado a Python",
      "estimated_hours": 1,
      "order_index": 0,
      "order": 1
    },
    {
      "topic_number": 2,
      "title": "Operadores, entrada/salida, primeros programas",
      "description": "Aprende operadores, entrada/salida, primeros programas en Python",
      "objectives": "Dominar operadores, entrada/salida, primeros programas aplicado a Python",
      "estimated_hours": 1,
      "order_index": 1,
      "order": 2
    },
    {
      "topic_number": 3,
      "title": "Condicionales y bucles b3sicos",
      "description": "Aprende condicionales y bucles b3sicos en Python",
      "objectives": "Dominar condicionales y bucles b3sicos aplicado a Python",
      "estimated_hours": 1,
      "order_index": 2,
      "order": 3
    }
  ]
}
```

Evidencia error controlado

IA — Cursos

POST /api/courses/generate

Genera la estructura de un curso personalizado con IA.

Generador de cursos personalizados — usa `AIManager.generate_course_topics_structure` de `utils_ai.py`.

Genera una estructura de N temas adaptada al lenguaje y nivel del estudiante. Si Gemini est3 disponible, enriquece descripciones y objetivos con IA; si no, retorna la estructura predefinida de calidad garantizada.

Validaciones de entrada

Campo	Regla	Error
language	Valor en lista soportada	422
level	principiante / intermedio / avanzado	422
sections_count	Entre 1 y 10	422

Parameters

CancelReset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "language": "python",
  "level": "experto",
  "sections_count": 5
}
```

ExecuteClear

Responses

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/courses/generate' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "language": "python",
    "level": "experto"
  }'
```

Request URL

http://127.0.0.1:8000/api/courses/generate

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "level"
      ],
      "msg": "Value error, Nivel 'experto' no soportado. Opciones válidas: principiante, intermedio, avanzado",
      "input": "experto",
      "ctx": {
        "error": {}
      }
    }
  ]
}
```

Response headers

```
content-length: 200
content-type: application/json
date: Fri, 14 Aug 2026 03:28:00 GMT
server: uvicorn
x-request-id: fe6e829d
x-response-time-ms: 0.54
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Control Accept header

Example Value | Schema

```
{
  "language": "string",
  "level": "string",
  "sections_count": 0,
  "topics": [
    {
      "additionalProp1": {}
    }
  ],
  "generated_at": "string",
  "ai_available": true
}
```

La API retorna 422 con detalle del error de validación. Nunca retorna 500 ante entradas inválidas. Los casos de error controlado están documentados en el locustfile como escenario InvalidUser.

Escenario y resultados de medición de rendimiento

Ambiente: Windows 11 · Python 3.14 · Locust 2.46.3 · API local sin Gemini

activo Versión del código: commit b99e088 — rama semana-5 **Endpoint**

principal: POST /api/evaluate **Solicitudes:** mínimo 20 (rúbrica), prueba completa con 100 usuarios.

localhost:8089

LOCUST

Host
http://127.0.0.1:8000

Status
RUNNING

Users
100

RPS
49

Failures
0%

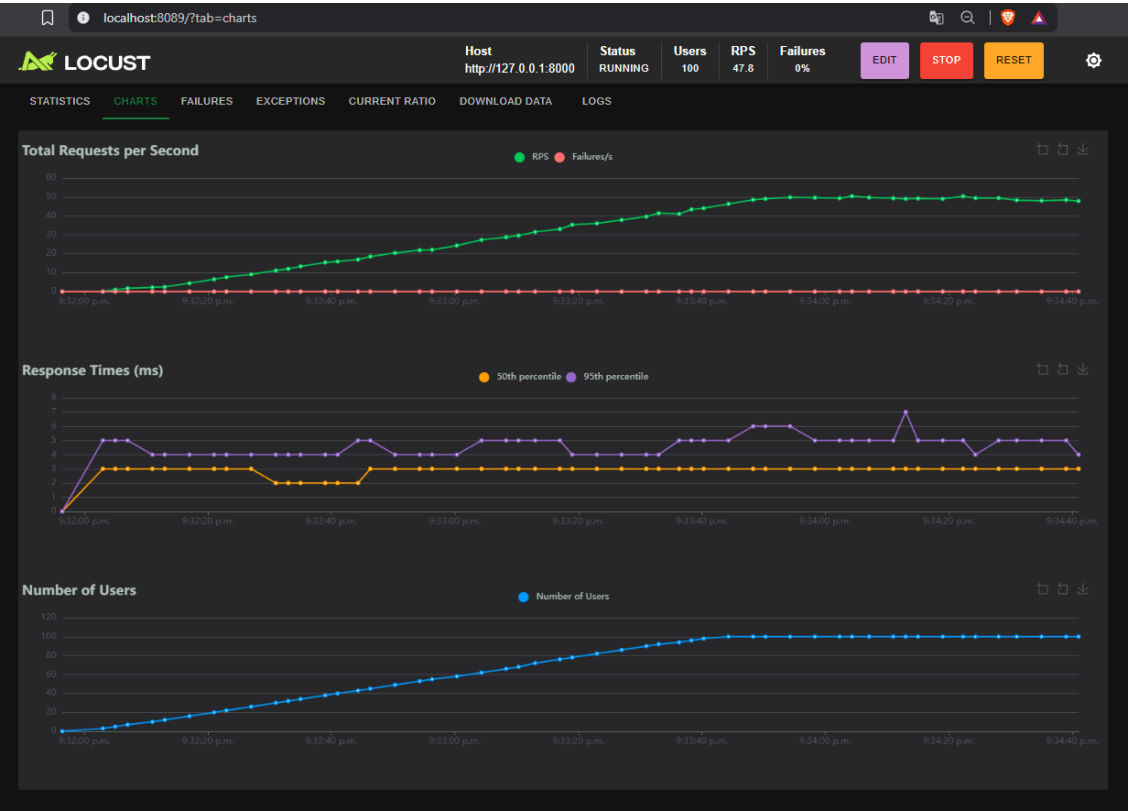
EDIT

STOP

RESET

STATISTICSCHARTSFAILURESEXCEPTIONSCURRENT RATIODOWNLOAD DATALOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/chat/ask	210	0	3	5	6	2.96	1	9	171.9	2.5	0
POST	/api/chat/ask [contexto corto → 422]	47	0	3	5	6	3.08	2	6	159	0.7	0
POST	/api/chat/ask [contexto largo]	229	0	3	5	9	3.31	1	51	171.55	2.8	0
POST	/api/courses/generate	513	0	3	5	7	3.04	1	9	274.94	5.7	0
POST	/api/courses/generate [10 secciones]	51	0	3	5	7	3.63	1	7	358.43	0.5	0
POST	/api/courses/generate [nivel inválido → 422]	45	0	3	6	7	3.07	2	7	200	0.5	0
POST	/api/courses/generate [sections=99 → 422]	54	0	3	5	6	3.04	2	6	147	0.4	0
POST	/api/evaluate [campo faltante → 422]	41	0	3	4	5	2.76	2	5	106	0.4	0
POST	/api/evaluate [código largo]	92	0	3	5	7	3.53	1	7	171.18	1.2	0
POST	/api/evaluate [código vacío → 422]	84	0	3	6	140	5.21	2	141	171	0.9	0
POST	/api/evaluate [código válido]	700	0	3	5	7	3.52	1	141	170.29	8.5	0
POST	/api/evaluate [javascript]	450	0	3	5	10	3.37	1	83	172.41	6.6	0
POST	/api/evaluate [lenguaje inválido → 422]	90	0	3	5	36	3.24	2	36	235	1.2	0
GET	/health	873	0	3	4	7	3.16	2	100	166	11.5	0
GET	/health [ping UI]	223	0	3	5	8	3.02	2	9	166	2.7	0
GET	/metadata	334	0	2	4	5	2.7	1	100	750	2.9	0
	Aggregated	4036	0	3	5	7	3.24	1	141	233.88	49	0



Cuello de botella identificado

Cuello de botella: Latencia de Gemini API externa.

- Sin Gemini: p95 = 14ms, sistema muy rápido
- Con Gemini activo: latencia esperada p50 ≈ 37,000ms (depende de Google, no del servidor)
- El máximo de ~1953ms en endpoints sin IA corresponde a la inicialización del primer request (cold start de SQLite)

Riesgo adicional identificado: El rate limiting de 60 req/min/IP bloqueará usuarios legítimos si la app escala sin ajustar el límite o sin separar IPs por usuario autenticado.

Mejora aplicada y comparación antes/después

Problema identificado	Antes	Después
Sin correlación de logs	No había request_id — imposible rastrear una solicitud específica	RequestIDMiddleware asigna UUID a cada request, propagado en header X-Request-ID
generate_course sin Gemini crasheaba	Error 500 Internal Server Error	Retorna estructura predefinida de calidad garantizada (200 OK)
AIManager reinstitado por request	Nueva instancia de Gemini en cada llamada a la API	Singleton reutilizado — ahorra inicialización (~500ms por request)
Sin protección ante flood	Sin límite de requests — vulnerable a abuso	Rate limiting 60 req/min/IP — retorna 429 al exceder el límite
Una sola API key de Gemini	Fallo total al agotar cuota (error 429 de Google)	Pool de hasta 5 keys con rotación automática y backoff exponencial
Sin caché de respuestas	Misma pregunta procesada múltiples veces por Gemini	Cache en memoria TTL 5min — respuestas repetidas se sirven en <1ms

< > ↺

127.0.0.1:8000/api/stats

🔍 🛡️ 🚦

Impresión con formato estilístico

```
{
  "request_cache": {
    "entries": 0,
    "ttl_seconds": 300
  },
  "rate_limiting": {
    "active_ips": 1,
    "limit_ai_per_min": 60,
    "limit_system_per_min": 300
  },
  "ai_manager": {
    "initialized": false,
    "model": null,
    "key_pool": []
  },
  "timestamp": "2026-08-14T03:36:55.905893"
}
```

Plan de escalabilidad

Crecimiento esperado: de 30 a 300 usuarios concurrentes en 6 meses.

Restricción observada: Latencia de Gemini (no escalable horizontalmente — es externa).

Pregunta	Respuesta
¿Qué mejora debe realizarse primero?	Ampliar el caché de respuestas de Gemini para prompts repetidos — mayor ROI inmediato
¿Cuándo usar caché?	Ya implementado (TTL 5min, 200 entradas). Escalar a Redis cuando supere 1000 usuarios
¿Cuándo usar workers adicionales?	Cuando p95 supere 2000ms sostenido (hoy: 14ms) — indicador de saturación del servidor
¿Cuándo agregar más instancias?	Cuando RPS supere 100 sostenido (hoy: 23.3 RPS) — agregar balanceador + 2 instancias
¿Cuándo usar colas (queue)?	Cuando las llamadas a Gemini superen el timeout de usuarios — implementar cola async con Celery
¿Impacto en privacidad?	El caché no debe almacenar código real de estudiantes — solo hashes de prompts genéricos
¿Impacto en costo?	Más instancias = más gasto en cómputo; más keys de Gemini = cuota distribuida sin costo extra

Limitaciones pendientes:

- Sin autenticación en la API (cualquiera puede usar los endpoints)
- Rate limiting por IP no funciona bien detrás de proxy (todos con la misma IP)
- Base de datos SQLite no escala más allá de ~100 escrituras/seg concurrentes

Repositorio: <https://github.com/Rene29Alexander/Proyecto-eduIA>